

Simple Bank: live demo script

Running ``python console.py --mongo`` beside the MongoDB Atlas Data Explorer

Before you start

Two windows, side by side. The terminal on the left, a browser on the right.

```
python console.py --mongo --reset
```

`--reset` empties the demo database so you begin from nothing, which makes the first record you create visibly the first record. Leave it off to keep what is already there.

LOOK: cloud.mongodb.com → Cluster0 → Browse Collections → the database named in the console banner (your `MONGODB_DB + _console`)

! Atlas does not refresh by itself. Every time you want to show a change, press its refresh icon. An empty-looking collection is almost always a stale page, not a failed write.

! Set `BANK_SECRET` in `.env` first, or tokens stop working whenever you restart the server. Not needed for the console, but it will bite you if you also demo the REST API.

SAY: “This is the same service layer the REST API calls. Nothing here can succeed that the API would refuse.”

1 — Register a user

Choose **1**. It prompts one field at a time. Let the room pick the name.

```
Name / Email / Password (not hidden) / User or Admin (1 or 2)
```

LOOK: `users` → the `_id` the console just printed

SAY: “The password is not in this document. There is a `password_hash` field — PBKDF2, salted, 600,000 rounds. The plaintext never reaches the database, and never reaches any response.”

LOOK: `counters` → `_id`: “users”

SAY: “That is our `AUTO_INCREMENT`. Mongo would give us an `ObjectId`; the brief asks for integer ids, so we allocate them atomically on the server with `find_one_and_update` and `$inc`.”

! Register a second user with the SAME email to show the refusal. It is stopped by a unique index on `users.email`, not by Python — so it holds even if two servers try at once.

3 — Open an account

Choose **3**. Pick Checking or Savings, then an opening balance. Type it in dollars, e.g. 250.00.

SAY: *"Watch the conversion line: 250.00 becomes 25000 cents. Money is an integer number of cents everywhere — never a float, never a Decimal. That is the single design decision I would defend hardest."*

! The console prints the ACCOUNT NUMBER in a box. That number is the `_id` of the accounts document, and every later prompt offers it as the default — press ENTER to take it.

LOOK: accounts → the new `_id`. Point at the balance field.

SAY: *"balance is 25000, an Int64. Not 250.00 as a Double. A float cannot hold 0.10 exactly and the error compounds across a ledger."*

LOOK: transactions → the `_id` printed alongside

SAY: *"The opening balance was not assigned, it was posted. Every balance in this system equals the sum of its ledger entries, and that is true from the first second the account exists."*

4 — Deposit (and the double-click)

Choose **4**. Press ENTER for the account number, type an amount, then press ENTER again to take the suggested reference id (**ref-001**).

SAY: *"A reference id is a label the caller invents for one transaction — a real client would send a UUID. The bank stores it and refuses anything that repeats it."*

LOOK: transactions → new `_id`, and **accounts** → the balance field has changed

SAY: *"Two documents changed together: the ledger entry and the balance. Never one without the other."*

Now do it again — same account, same amount — but this time type **ref-001** back in instead of pressing ENTER. The console lists the ids already used, so it is on screen in front of you.

SAY: *"Refused. That is the double-clicked submit button. The guarantee is a unique index on client_txn_id, so it holds across processes — a set in Python memory would only protect one server."*

LOOK: Refresh **transactions** and count: no new document. The refusal wrote nothing.

! This is the most convincing thing in the demo. A database with nothing new in it, after an action that looked like it should have added something.

5 — Withdraw more than the balance

Choose **5** and ask for more than the account holds.

SAY: *"InsufficientFunds. Over HTTP that is a 409. The check and the write happen inside one transaction, so two simultaneous withdrawals cannot both pass it."*

LOOK: accounts → the balance is unchanged. Nothing was written.

2 — Log in as someone else

Register a second user (**1** again), then use **2** to move between them.

SAY: “Logging in writes nothing — the console says so. It reads the stored hash and compares. Even an unknown email is checked against a decoy hash, so the response takes the same time either way and does not leak which addresses are registered.”

Now, as the second user, try **7** on the FIRST user’s account number — type it over the default rather than pressing ENTER. The prompt accepts any number on purpose.

SAY: “AccountNotFound — and note it says no account with id 1, the same thing you get for an id that was never issued. Over HTTP that is a 404, not a 403: saying ‘you are not allowed to see this’ would confirm the account exists.”

Try **4** on the same account number for the same effect on a write.

LOOK: Nothing changed in **transactions** or **accounts**.

6 — Transfer between accounts

Open an account for the second user first, then choose **6**.

LOOK: **transactions** — TWO new documents, TRANSFER_OUT and TRANSFER_IN. **accounts** — both balances moved.

SAY: “Four documents, one MongoDB transaction. Atlas can never show you a state where the money has left one account and not arrived at the other.”

SAY: “This is why we are on Atlas rather than a local mongod. Multi-document transactions need a replica set. A standalone server accepts the same code and gives no atomicity, silently.”

8 — Admin actions

Register an ADMIN (option **1**, then choose 2 at the role prompt), then choose **8**.

Freeze an account. Try a reason under ten characters first — it is refused.

LOOK: **accounts** → status is FROZEN. **audit_log** → the `_id` printed — who did it, and the reason.

Switch back to that customer (**2**) and try to deposit.

SAY: “Refused while frozen.”

Then post an adjustment.

LOOK: **transactions** → the new entry carries **adjusted_by** and **reason**; an ordinary deposit has neither.

SAY: “There is no `set_balance` method anywhere in this codebase. It is the easiest admin feature to write and the wrong one: a balance that moves without a ledger entry breaks the invariant permanently, and afterwards you cannot tell which of the two is right.”

Finish with **8** → **Reconcile every account**.

SAY: “Every balance still equals the sum of its ledger. Integers, so that comparison is exact and needs no tolerance.”

7 and 9 — Reads, and the closing proof

7 shows history and reconciliation for one account. It writes nothing, and says so.

9 opens a second, independent connection and reads everything back.

SAY: "None of those numbers came from this program's objects. They were read out of Atlas by a connection that has never seen them."

Strongest close: quit the console entirely, start it again **without --reset**, and press 9.

SAY: "Different process. Same data. That is the thing an in-memory store can never show you."

If something goes wrong

! The collection looks empty. Press refresh in Atlas. It does not poll.

! Nothing connects. Run `python tools/check_mongo.py` — seven checks, each naming what to change. Usually the IP access list.

! 'email already registered' for an email that is not there. The id counter is behind the data. Reload with `python server.py --reset`.

! A refusal you did not plan. Keep going and read it out. Every refusal in this system is a rule doing its job, and they are the most interesting thing on the screen.

Also available

? at the console menu — every term on screen, with the reason behind it: the five collections, `_id`, reference id, cents, and what each refusal proves. Read it before you present; open it live if a question lands that you would rather answer from the screen than from memory.

`python demo.py --step` — the scripted walkthrough, paused between sections. Use it if you would rather not type.

`python simulate.py` — continuous traffic against Atlas, about one action every two seconds. Good for leaving running in the background while you talk.

`python server.py` then Postman — the same rules as real HTTP status codes, using the committed collection.